이재혁

백엔드 개발자

연락처: (보류)

요약

- 4년간 약 300개 고객사, 월 최대 9천만 건을 처리하는 B2B 메시징 플랫폼을 설계부터 운영까지 담당했습니다.
- 20년 된 레거시를 Spring Cloud MSA로 전환하는 프로젝트의 개발 리더를 맡아 서버를 **102대에서 23대**로 줄이고 처리량은 26% 높이며 전송 비용은 30% 절감했습니다.
- 회원, 인증, 결제 데이터 통합과 Admin 운영 도구, 무중단 배포까지 메시징 플랫폼 운영에 필요한 전 영역을 경험했습니다.

About Me

4년 가까이 월 최대 9천만 건, 약 300개 고객사의 문자를 처리하는 메시징 플랫폼을 맡아 왔습니다. 20년 된 C, Java, Python 레거시를 운영하다가 Spring Cloud MSA로 다시 만드는 차세대 프로젝트에서 개발 리더를 맡았고, 레거시를 정리해 물리 서버를 102대에서 23대로 줄이는 한편 라우팅을 바꿔 처리량은 26% 높이고 전송 비용은 30% 낮췄습니다. 새벽에만 하던 배포도 롤링 배포로 바꿔 운영 시간에 할 수 있게 됐습니다.

기술을 고를 때는 왜 이 구조여야 하는지를 먼저 따지고, 그 판단이 맞는지는 부하 테스트로 확인하는 편입니다. WebFlux는 MVC와 같은 조건에서 비교한 뒤에 도입했고, 농협 RCS 서버는 4코어 서버 한 대로 3,000 TPS를 CPU 사용률 21% 이내로 처리했습니다.

요즘은 AI 에이전트에게 API 설계부터 테스트, PR 생성까지 맡겨 보면서, 규칙 문서와 git 훅을 걸어 검증을 거치지 않은 코드는 들어가지 못하게 해두고 쓰고 있습니다.

경력 개요

서울이동통신(주)

2024.XX ~ 2025.06 (임시, 수정 필요)

백엔드 개발자 / 팀장

스탠다드 네트워스(주)

2021.10 ~ 2024.XX (임시, 수정 필요)

백엔드 개발자 / PL

기술 스택

Language / Framework: Java, Spring Boot, Spring WebFlux, Spring Cloud, Netty

Database: MariaDB, PostgreSQL, Oracle, Redis, JDBC, R2DBC, JPA

Messaging: RabbitMQ

DevOps / Tools: GitHub Actions, Prometheus, Grafana, k6, Jira

학력 및 자격

정보처리기사, 2025.12

중앙정보기술인재개발원 Java 응용 SW 개발자 전문과정, 2020.02 ~ 2020.08

한림성심대학교 인터넷비즈니스과 졸업, 2015.02 ~ 2019.02

서울이동통신(주)

백엔드 개발자 / 팀장 | 2024.XX ~ 2025.06 (임시, 수정 필요)

a015 회원 통합 및 서비스 개선

XXXX.XX ~ XXXX.XX (수정 필요)

015 번호를 기업과 개인이 발급받아 문자 송수신, 스마트 ARS, 콜센터 기능을 이용할 수 있는 서비스입니다. 프로젝트 PM으로 일정과 프로젝트 관리, 기능 설계를 담당하며 개발에도 직접 참여했습니다.

팀장 1명(본인), 프론트엔드 2명, 백엔드 3명, 기획 1명, 마케팅 1명

Java 17, Spring Boot 3.x, JDBC, PostgreSQL, Oracle

3개 사이트 회원 통합 및 데이터 이관

- 신규 서비스 'a015'의 회원 확보를 위해 기존 운영되던 '아레오', '네티앙'의 회원, 결제, 문자 내역, 캐시 등 **약 5천만 row** 데이터를 Batch Query를 작성하여 Oracle → PostgreSQL로 이관 및 통합했습니다.
- '싸다오' 회원은 회원 정보에 CI 값이 존재하지 않아, 소셜 로그인 기능을 구현하여 본인 인증을 통한 CI 값으로 회원 데이터를 이관했습니다.

채팅방 첨부파일 조회 구조 개선 및 응답 성능 최적화

- 채팅방 진입 시 메시지 조회 API에서 내역과 이미지 원본을 함께 반환하는 구조로 인해, 현재 화면에 표시되지 않는 이미지까지 초기 응답에 포함되어 데이터 전송량이 증가하고 화면 표시가 지연되는 문제를 확인했습니다.
- 메시지 조회 API는 내역과 첨부파일 메타데이터만 반환하도록 변경하고, 별도의 파일 다운로드 API를 제공해 화면에 표시되는 첨부파일만 개별로 조회하도록 개선했습니다.
- 채팅방 초기 데이터 Payload를 **12MB → 1.8MB**로 **85% 감소**시키고, 100Mbps 환경의 성능 테스트에서 초기 콘텐츠 수신 완료 시간의 p95를 **1,122ms → 186ms**로 **83% 단축**했습니다.

팀 개발 및 협업 프로세스 개선

- Jira 티켓 기반 업무와 Slack 주제별 채널을 도입해, 개별 요청으로 공유되지 않던 일정과 이슈, 논의 기록을 팀 전체가 한곳에서 보도록 했습니다.
- 프론트엔드와 백엔드의 서로 다른 Git 관리 규칙이 신규 개발자의 온보딩과 유지보수에 혼란을 줄 수 있다고 판단하여 Git Flow 브랜치 전략을 적용했습니다.

문자 메시징 플랫폼의 레거시 운영부터 차세대 MSA 전환까지, 설계, 개발, 성능 검증, 운영 전환을 담당했습니다.

차세대 통합 메시징 플랫폼 구축

2023.04 ~ 2025.06

20년간 SMS, LMS, MMS, RCS, 알림톡 전송 서비스를 제공해 온 B2B SaaS 플랫폼입니다. 플랫폼의 개발 비용과 운영 비용을 줄이고 성능과 확장성을 높이기 위해 Polyglot 환경을 하나로 통합하는 프로젝트에 개발 리더로 참여했습니다.

백엔드 5명, 프론트엔드 2명

Java 17, Spring Boot 3.x, Spring WebFlux, Spring Cloud, R2DBC, MariaDB, Redis, RabbitMQ, k6

레거시 플랫폼 통합과 MSA 전환

- Polyglot(C, Java, Python) 레거시의 소스 코드와 기능 명세서, 개발 일지를 분석한 내용과 유관부서 회의를 주관해 수집한 요구사항을 바탕으로 공통 기능과 문자 유형별 기능을 분류해, 필요한 서비스만 따로 개발하고 배포할 수 있는 Java/Spring Cloud 기반 MSA로 재구성했습니다.
- 발신번호 검증, 이력 관리 등 환경마다 따로 개발하던 공통 기능을 서비스 단위로 통합하여, 개발 및 유지보수 비용을 줄였습니다.
- 서비스를 통합하면서 기존 레거시 시스템을 정리하여 물리 서버 운영 규모를 **102대 → 23대**로 감축했습니다.
- 서버마다 따로 관리하던 설정 파일을 Config Server로 중앙 관리하여, 옵션 변경 시 모든 서버의 파일을 일일이 수정하던 작업을 줄였습니다.
- 회원 정보, 서비스 이력, 라우팅, 관리자 등 플랫폼 전반의 DB를 설계하고, 월 9천만 건 규모의 발송 이력 테이블은 서비스 요청일 기준 월별 파티셔닝으로 구성해 조회 시 해당 기간의 파티션만 읽도록 했습니다.

B2B 메시징 서비스 RESTful API 개발

- 고객사 환경마다 Agent 설치와 커스터마이징이 필요했던 기존 연동 방식을 개선하고자 SMS, LMS, MMS, RCS, 알림톡을 제공하는 RESTful API를 설계하고 개발해, 고객사가 전용 프로토콜 구현 없이 API 계정 발급과 JWT 인증만으로 서비스 연동을 할 수 있게 했습니다.
- 서버 증설이나 배포 중에도 서비스가 끊기지 않도록, Spring Cloud Gateway의 LoadBalancer로 정상적인 서버 인스턴스로만 요청이 분산되도록 구현했습니다.
- I/O가 많은 문자 발송 API에 맞는 처리 모델을 찾기 위해 MVC와 WebFlux를 비교한 뒤 WebFlux를 도입했으며, MVC 대비 처리량 **4,449 TPS → 4,820 TPS**로 8.3% 증가, 응답속도 p95 **943ms → 641ms**로 32%, p99 **1,625ms → 797ms**로 51% 개선을 확인했습니다.
- Spring Security RSA/JWT 기반의 회원 인증을 설계 및 구현했습니다.
- 고객사별 요청 허용량은 RedisRateLimiter의 Token Bucket으로 제한해 과도한 요청으로부터 서비스를 보호하고, 메시지 중복 전송 요청을 Redis SET NX 기반의 멱등키 검증으로 차단했습니다.
- MMS, RCS, 알림톡 첨부파일의 업로드와 다운로드 API를 리액티브 스트리밍으로 구현해 버퍼 단위로 처리함으로써 대용량 파일에서도 Heap 사용량이 늘지 않도록 하고, 다운로드는 Zero-Copy로 커널에서 소켓으로 바로 전송해 메모리 복사와 CPU 부하를 줄였습니다.
- REST API로 연동한 고객사는 기존 Agent 방식과 달리 설치와 현장 지원이 필요 없어져, 2024.02 ~ 2025.05 연동 관련 **VOC 0건**을 기록했습니다.

차세대 통합 메시징 플랫폼 구축

RabbitMQ 기반 비동기 이벤트 처리 설계

- 서비스 간 직접 호출을 줄이고 라우팅과 이력 저장을 독립적으로 수행할 수 있도록, Topic Exchange와 서비스별 Queue를 통해 한 번 발행한 이벤트를 각 서비스가 받아 처리하는 구조를 설계했습니다.
- 메시지가 브로커에 도달하지 못하거나 처리 도중 서버가 종료되어 유실되지 않도록, Publisher Confirm으로 브로커 수신을 확인하고 비즈니스 로직 완료 후 Manual ACK를 전송했습니다.
- 소비자가 처리량보다 많은 메시지를 받지 않도록 서비스별 Prefetch를 조정하고, 실패 메시지는 DLX/DLQ로 분리해 반복 처리가 신규 요청을 지연시키지 않도록 했습니다.
- 첨부파일 바이너리가 이벤트에 실려 RabbitMQ에 부하를 주지 않도록, 파일은 Storage에 저장하고 이벤트에는 식별 Key만 전달하는 Claim Check 패턴을 적용했습니다.
- 비동기 환경에서는 이력 갱신 이벤트가 생성 이벤트보다 먼저 도착할 수 있어, DB Logger에서 R2DBC 기반 Upsert로 이벤트 순서와 관계없이 이력을 반영하도록 했습니다.

라우팅 개선을 통한 처리량 증가와 전송 비용 절감

- 처리량이 서로 다른 Task Queue들에 라운드로빈 방식으로 균등하게 작업을 분배해 일부 Queue에 병목이 발생하던 문제를 각 Queue의 처리량에 비례해 작업을 분배하는 가중치 방식으로 바꿔 전체 **처리량을 26% 개선**했습니다.
- 외부망마다 다른 계약 물량과 전송 단가가 경로 선택에 반영되지 않던 문제는 Redis에 저장한 외부망별 쿼터와 단가를 참조해 계약 물량을 우선 충족한 뒤 잔여 물량을 낮은 단가의 경로로 배분하도록 해 **전송 비용을 약 30% 절감**했습니다.

CI/CD 자동화 파이프라인 구축

- 개인 PC, SVN, 물리 서버에 분산돼 있던 소스와 빌드 파일을 GitHub와 릴리스 태그로 관리해 운영 버전을 추적할 수 있게 했습니다.
- 수동으로 파일을 올리고 재기동하던 배포를 사내망 self-hosted runner와 GitHub Actions, Shell Script로 자동화했습니다.
- 테스트에 실패하면 배포를 중단하고, 배포에 실패하면 릴리스 태그로 이전 버전을 재배포할 수 있게 했습니다.
- 기존 레거시에서 서버 작업 시 다운타임이 발생하여 서비스 중지 시간에 맞춰야 했던 배포를 Graceful Shutdown과 헬스 체크 기반의 롤링 배포로 전환해 **서비스 중단 없이 배포**할 수 있도록 했습니다.

운영 도구와 모니터링 구축

- JPA, Thymeleaf, JavaScript로 계정별 접근 권한을 갖춘 Admin Web을 개발해, 운영팀과 영업팀이 고객사 정보, 발송 이력, 서비스 이용 추이, 통신사 발송 경로 등 서비스 상태를 한눈에 파악할 수 있게 했습니다.
- 영업팀의 문자 라우팅 변경 요청을 운영팀이 직접 코드를 수정하던 방식에서 Admin 화면을 통해 즉시 처리할 수 있도록 업무 프로세스를 개선했습니다.
- 담당자가 모니터링 화면을 보고 위험을 감지하던 흐름을 시스템이 담당자에게 알리는 흐름으로 바꾸고자, Prometheus와 Grafana로 하드웨어, 애플리케이션, RabbitMQ, Redis, DB 등 서비스 전반의 상태를 수집하고, 장애나 임계치 초과 시 Alertmanager로 Slack에 전파하도록 구성했습니다.

농협정보시스템 RCS 전송 서버 개발

2022.01 ~ 2023.03

농협정보시스템의 월 400만 건 규모 SMS, MMS, RCS 문자 전송을 처리하기 위한 서비스를 구축하는 프로젝트이며, 그중 RCS 전송 서버의 설계와 개발을 담당했습니다.

백엔드 3명

Java 11, Spring Boot 2.7, Netty 4.1, RabbitMQ, Redis, MariaDB, Prometheus, Grafana

RCS 전송 시스템 설계

- 고객사 클라이언트 프로토콜 규격의 TCP 통신으로 최대 3,000 TPS의 RCS 메시지를 처리해야 하는 요구사항에 맞춰, 요청 수신부터 전문 생성, 이력 저장, 첨부파일 처리, 외부망(이동통신사) 전송과 이후 비동기로 수신되는 RCS 결과 회신까지 전체 흐름을 설계하고 구현했습니다.
- 고객사 요청 세션과 외부망 전송 세션은 각각 따로 증설될 수 있어, 두 흐름을 직접 연결하지 않고 RabbitMQ 이벤트로 분리해 수신, 전송, 결과 회신, 이력 저장 서버가 서로 영향 없이 독립적으로 확장되도록 설계했습니다.

RCS 서버 개발

- 고객사의 RCS 전송 요청을 수신하는 Receiver와 처리 결과를 고객사에 회신하는 Reporter는 고객사와 각 240개 TCP 세션을 상시 유지하는 서버로, 세션이 증설되어도 연결마다 할당되는 스레드와 메모리가 함께 늘어나지 않도록 적은 수의 스레드로 다수의 연결을 논블로킹 처리하는 Netty의 EventLoop 모델을 적용했습니다.
- 고객사 전문 규격의 송수신은 Netty Codec으로 구현해, 스트림 기반인 TCP는 전문 단위 수신을 보장하지 않으므로 Receiver는 Decoder에서 Header의 길이 정보를 기준으로 전문을 조립하고, Reporter는 Encoder에서 처리 결과를 고객사 전문 규격의 바이너리로 변환해 회신하도록 했습니다.
- 고객사의 동일 요청이 중복 발송되지 않도록 Redis SET NX EX로 차단하고, 이벤트 발행에 실패하면 Key를 삭제해 재전송 요청이 정상 처리되도록 했습니다.
- 비동기 환경에서는 이력 갱신 이벤트가 생성 이벤트보다 먼저 도착할 수 있어, DB Logger에서 JDBC 기반 Upsert로 이벤트 순서와 관계없이 이력을 반영하도록 했습니다.
- 전송 요청 수신부터 처리 결과 회신까지 4코어 8스레드 서버 1대로 요구 처리량인 **3,000 TPS를 p95 52ms, 오류율 0%, 최대 CPU 사용률 21%**로 처리했으며, JMC를 통해 Heap 메모리 사용량이 안정적으로 유지되는 것을 확인했습니다.

RabbitMQ를 활용한 메시지 유실 방지

- Publisher Confirm과 Manual ACK를 적용해 발행과 소비 단계의 이벤트 유실을 방지했습니다.
- 이동통신사 응답 시간 초과로 재처리가 필요한 건이 정상 발송 건을 지연시키지 않도록, NACK로 DLQ에 분리해 재처리했습니다.

모니터링 구축

- 하드웨어와 JVM부터 TCP 세션, 흐름별 처리량, 응답 속도까지 Prometheus로 수집하고, 사내 운영팀이 서비스 상태를 확인할 수 있는 Grafana 대시보드를 구성했습니다.